END-TO-END DATA ANALYTICS PROJECT

Sample Superstore: Python + SQL + Data Modelling + Power BI

A practical, implementation-ready project handbook

What this guide contains

- Project scope, business questions, dataset preparation and folder structure
- Complete Python workflow for profiling, cleaning, transformation and exports
- SQLite database design, primary/foreign keys, loading and analytical SQL
- Dimensional modelling, star-schema concepts and Power BI relationships
- DAX measures, dashboard page designs, slicers, drill-through and validation
- Project documentation, insight-writing template, QA checklist and interview story

Version: 1.0 | Prepared: 4 November 2026

Important: the guide uses the commonly distributed Sample Superstore column layout. Check your actual file's column names and adjust the input paths if your copy differs. No dataset was attached when this guide was prepared.

How to use this handbook

Follow the chapters in order. Do not jump directly to dashboard design: the quality of the Power BI report depends on correct data grain, keys, dates and relationships. The code is designed for a typical Superstore Orders file and an optional Returns file.

• Complete each phase and check its output before moving on.

• Keep the original source files unchanged. Work on copies and write processed outputs to the output folder.

• Run the Python script from the project root so relative paths resolve correctly.

• Use SQLite for a beginner-friendly local SQL database. The same dimensional concepts transfer to SQL Server, PostgreSQL or cloud warehouses.

• Build and save the final .pbix file in Power BI Desktop. This PDF gives the model, measures and report specification; it is not itself a PBIX file.

Contents

1. Project overview and business problem
2. Dataset, grain and business questions
3. Tools and installation
4. Folder structure and setup
5. Python: complete data preparation script
6. Data model and table definitions
7. SQL: database creation, loading and analysis
8. Power BI: import and relationships
9. DAX measures library
10. Dashboard pages and visual specifications
11. Interactions, usability and design
12. Validation and testing
13. Business insights and recommendations
14. Documentation and deliverables
15. Interview explanation and resume bullets
16. Troubleshooting and final checklist

1. Project overview and business problem

Project title: Retail Sales, Customer, Product and Returns Analysis using Python, SQL and Power BI.

Business scenario: A retail business wants to understand sales performance, profitability, customer behaviour, product performance, shipping patterns and returned orders. The analyst receives transactional Excel/CSV files and must create a reliable analytical dataset and interactive report.

Objectives

• Convert raw transactional data into clean, consistent, analysis-ready tables.

• Build a relational database with appropriate primary and foreign keys.

• Create a dimensional model that supports fast, understandable Power BI analysis.

• Measure sales, profit, order volume, average order value, discounts, customer behaviour and returns.

• Communicate findings through a multi-page interactive Power BI report.

Project workflow

Phase	Work	Output
1. Understand	Inspect files, columns, grain, business definitions	Data dictionary and scope
2. Prepare	Profile, clean, standardise and validate in Pandas	Clean CSVs and quality report
3. Model	Create dimensions and fact tables	Dimensional model
4. Store	Create SQL tables, keys and load data	SQLite database and SQL queries
5. Report	Connect Power BI, relationships and DAX	Interactive PBIX report
6. Communicate	Interpret results and document limitations	README, insights and presentation

2. Dataset, grain and business questions

Expected source files

The common Sample Superstore dataset contains an Orders sheet/file with one record per order line. Some versions also provide a Returns file with an order-level Returned flag. Typical columns are listed below; your copy may use slightly different names.

Column	Meaning	Typical type
Row ID	Unique sales line identifier	Integer
Order ID	Order identifier; may repeat across lines	Text
Order Date / Ship Date	Order and shipment dates	Date
Ship Mode	Shipping service level	Category
Customer ID / Name	Customer identifier and display name	Text
Segment	Customer segment	Category
Country, City, State, Region	Geography	Text
Product ID / Name	Product identifier and description	Text
Category / Sub-Category	Product hierarchy	Category
Sales, Quantity, Discount, Profit	Transaction measures	Numeric

Critical concept: grain

Fact_Sales grain: one row per sales order line, identified by Row ID (or another genuine unique line key). An order can contain several products, so Order ID is not a unique sales-line key.

Fact_Returns grain: one row per returned order if the Returns file only contains Order ID and Returned. Do not manufacture product-level return records from an order-level flag.

Why grain matters: counting rows in Fact_Sales counts lines, not orders. Use DISTINCTCOUNT(Order_ID) for order count.

Business questions

• What are total sales, total profit, profit margin and order count?

• How do sales and profit change by month, quarter and year?

• Which regions, states and customer segments contribute most?

• Which categories and sub-categories generate profit or losses?

• Which products are top sellers and which are loss-making?

• How do discounts relate to profit?

• Which customers buy repeatedly and who are the highest-value customers?

• What percentage of orders are returned, and how does that vary by segment or region?

• Which shipping modes are associated with higher sales or longer fulfilment time, if dates support it?

Returns limitation: when the Returns file only flags an Order ID, you can analyse returned orders and return rate, but you cannot accurately attribute a return to a particular product line or calculate refund cost without line-level return/refund data.

3. Tools and installation

Tool	Purpose
Python 3.10+	Data preparation and automation
Pandas / NumPy	Data manipulation and numeric operations
OpenPyXL	Read Excel workbooks
SQLite	Local relational database for SQL practice
SQLAlchemy (optional)	Database connection abstraction
Power BI Desktop	Data model, DAX and interactive reports
VS Code or Jupyter	Write and run Python code

Install Python libraries

```
python -m venv .venv

# Windows:
.venv\\Scripts\\activate

# macOS / Linux:
source .venv/bin/activate

python -m pip install --upgrade pip
pip install pandas numpy openpyxl matplotlib seaborn sqlalchemy
```

Power BI Desktop is installed separately. If you use a Mac, Power BI Desktop is not natively available as a full desktop authoring app; use a Windows environment or an organisation-supported virtual desktop.

Suggested source files

• SampleSuperstore.xlsx (or Orders.csv)

• Returns.xlsx (optional; may be a separate workbook)

• Do not edit source files manually after starting the project.

4. Folder structure and setup

```
superstore-analytics/
■
■■■ data/
■   ■■■ raw/
■   ■   ■■■ SampleSuperstore.xlsx
■   ■   ■■■ Returns.xlsx
■   ■■■ processed/
■       ■■■ dim_date.csv
■       ■■■ dim_product.csv
■       ■■■ dim_customer.csv
■       ■■■ dim_order.csv
■       ■■■ fact_sales.csv
■       ■■■ fact_returns.csv
■
■■■ scripts/
■   ■■■ 01_profile_data.py
■   ■■■ 02_prepare_data.py
■   ■■■ 03_load_sqlite.py
■
■■■ database/
■   ■■■ superstore.db
■
■■■ output/
■   ■■■ data_quality_report.txt
■
■■■ powerbi/
■   ■■■ Superstore_Analytics.pbix
■
■■■ sql/
■   ■■■ analysis_queries.sql
■
■■■ docs/
■   ■■■ data_dictionary.md
■   ■■■ data_model.md
■   ■■■ insights.md
■
■■■ README.md
```

Create these folders before running the scripts. Keep raw, processed and output data separate. Add the .venv folder and any private/raw data to .gitignore if you use Git.

5. Python: complete data preparation

5.1 Read and profile the source

```
# scripts/01_profile_data.py
from pathlib import Path
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
ORDERS_FILE = ROOT / "data" / "raw" / "SampleSuperstore.xlsx"

# For a CSV source, use:
# df = pd.read_csv(ROOT / "data/raw/Orders.csv")
df = pd.read_excel(ORDERS_FILE, sheet_name=0)

print("Shape:", df.shape)
print("\\nColumns:", df.columns.tolist())
print("\\nData types:\\n", df.dtypes)
print("\\nMissing values:\\n", df.isna().sum().sort_values(ascending=False))
print("\\nDuplicate full rows:", df.duplicated().sum())
print("\\nSample:\\n", df.head())
print("\\nNumeric summary:\\n", df.describe(include="all").T)
```

Run this first. Confirm the worksheet name, column names, date formats and whether Row ID is unique. If your workbook has multiple sheets, inspect `pd.ExcelFile(path).sheet_names`.

5.2 Complete preparation script

This script normalises common column names, validates required fields, removes exact duplicate rows, creates dimensions and facts, builds a continuous date dimension and exports CSV files. It deliberately stops when required columns are missing rather than silently guessing.

```
# scripts/02_prepare_data.py
from pathlib import Path
import pandas as pd
import numpy as np

ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "data" / "raw"
OUT = ROOT / "data" / "processed"
REPORT = ROOT / "output" / "data_quality_report.txt"
OUT.mkdir(parents=True, exist_ok=True)
REPORT.parent.mkdir(parents=True, exist_ok=True)

ORDERS_PATH = RAW / "SampleSuperstore.xlsx"
RETURNS_PATH = RAW / "Returns.xlsx"  # optional

def normalise_columns(df):
    df = df.copy()
    df.columns = (
        df.columns.astype(str).str.strip()
          .str.replace(r"\\s+", " ", regex=True)
    )
    return df

def require_columns(df, required, label):
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"{label} is missing required columns: {missing}")

# 1) Load
orders = pd.read_excel(ORDERS_PATH, sheet_name=0)
orders = normalise_columns(orders)

# 2) Validate source schema
required = [
    "Row ID", "Order ID", "Order Date", "Ship Date", "Ship Mode",
    "Customer ID", "Customer Name", "Segment", "Country", "City",
    "State", "Region", "Product ID", "Product Name", "Category",
    "Sub-Category", "Sales", "Quantity", "Discount", "Profit"
]
require_columns(orders, required, "Orders")

# 3) Clean text and types
text_cols = orders.select_dtypes(include="object").columns
for col in text_cols:
    orders[col] = orders[col].astype("string").str.strip()

orders["Order Date"] = pd.to_datetime(orders["Order Date"], errors="coerce")
orders["Ship Date"] = pd.to_datetime(orders["Ship Date"], errors="coerce")
for col in ["Sales", "Quantity", "Discount", "Profit"]:
    orders[col] = pd.to_numeric(orders[col], errors="coerce")

# 4) Quality checks before removing records
quality = []
quality.append(f"Raw rows: {len(orders)}")
quality.append(f"Exact duplicate rows: {orders.duplicated().sum()}")
quality.append("Missing values by column:")
quality.append(orders.isna().sum().to_string())

# Exact duplicate removal is safe to consider; do not blindly deduplicate
# using Product ID or Customer ID because those keys repeat in transactions.
orders = orders.drop_duplicates()

# Row ID should identify each sales line in the standard dataset.
if orders["Row ID"].duplicated().any():
    dup_count = int(orders["Row ID"].duplicated().sum())
    raise ValueError(f"Row ID is not unique; investigate {dup_count} duplicate IDs.")

# Remove records that cannot be linked to core entities or dated.
required_non_null = ["Row ID", "Order ID", "Customer ID", "Product ID", "Order Date"]
before = len(orders)
orders = orders.dropna(subset=required_non_null)
quality.append(f"Rows removed for missing key/date fields: {before - len(orders)}")

# Do not automatically replace missing Sales/Profit with zero.
# Review missing numeric values; remove only if business-approved.
numeric_missing = orders[["Sales", "Quantity", "Discount", "Profit"]].isna().sum()
quality.append("Missing numeric values after key filtering:")
quality.append(numeric_missing.to_string())
```

```
# Optional validation rules
if (orders["Quantity"].dropna() < 0).any():
    quality.append("WARNING: negative Quantity values found; inspect source.")
if (orders["Discount"].dropna() < 0).any() or (orders["Discount"].dropna() > 1).any():
    quality.append("WARNING: Discount outside expected 0-to-1 range; verify source.")

# 5) Build dimensions
dim_product = (
    orders[["Product ID", "Product Name", "Category", "Sub-Category"]]
    .drop_duplicates()
    .rename(columns={
        "Product ID": "Product_ID", "Product Name": "Product_Name",
        "Sub-Category": "Sub_Category"
    })
)

# Detect conflicting product attributes rather than hiding them.
if dim_product["Product_ID"].duplicated().any():
    conflicts = dim_product[dim_product["Product_ID"].duplicated(keep=False)]
    raise ValueError("Conflicting product attributes for Product ID(s):\\n"
                     + conflicts.to_string(index=False))

dim_customer = (
    orders[["Customer ID", "Customer Name", "Segment", "Country",
            "City", "State", "Region"]]
    .drop_duplicates()
    .rename(columns={
        "Customer ID": "Customer_ID", "Customer Name": "Customer_Name"
    })
)
if dim_customer["Customer_ID"].duplicated().any():
    conflicts = dim_customer[dim_customer["Customer_ID"].duplicated(keep=False)]
    raise ValueError("Customer attributes vary across rows. Decide whether to "
                     "retain latest/current attributes or create a historical "
                     "dimension. Conflicts:\\n" + conflicts.to_string(index=False))

dim_order = orders[["Order ID"]].drop_duplicates().rename(
    columns={"Order ID": "Order_ID"}
)

# 6) Build continuous calendar dimension
min_date = orders["Order Date"].min().normalize()
max_date = max(orders["Order Date"].max().normalize(),
               orders["Ship Date"].max().normalize())
calendar = pd.DataFrame({"Date": pd.date_range(min_date, max_date, freq="D")})
calendar["Date_Key"] = calendar["Date"].dt.strftime("%Y%m%d").astype("int64")
calendar["Year"] = calendar["Date"].dt.year
calendar["Quarter"] = "Q" + calendar["Date"].dt.quarter.astype(str)
calendar["Month_Number"] = calendar["Date"].dt.month
calendar["Month_Name"] = calendar["Date"].dt.strftime("%B")
calendar["Year_Month"] = calendar["Date"].dt.strftime("%Y-%m")
calendar["Day"] = calendar["Date"].dt.day
calendar["Day_Name"] = calendar["Date"].dt.day_name()
dim_date = calendar[[
    "Date_Key", "Date", "Year", "Quarter", "Month_Number",
    "Month_Name", "Year_Month", "Day", "Day_Name"
]]

# 7) Build sales fact at order-line grain
fact_sales = orders[[
    "Row ID", "Order ID", "Order Date", "Ship Date", "Ship Mode",
    "Customer ID", "Product ID", "Sales", "Quantity", "Discount", "Profit"
]].copy()

fact_sales = fact_sales.rename(columns={
    "Row ID": "Sales_ID", "Order ID": "Order_ID",
    "Order Date": "Order_Date", "Ship Date": "Ship_Date",
    "Ship Mode": "Ship_Mode", "Customer ID": "Customer_ID",
    "Product ID": "Product_ID"
})
fact_sales["Order_Date_Key"] = pd.to_datetime(
    fact_sales["Order_Date"]).dt.strftime("%Y%m%d").astype("int64")
fact_sales["Ship_Date_Key"] = pd.to_datetime(
    fact_sales["Ship_Date"], errors="coerce").dt.strftime("%Y%m%d")
fact_sales["Ship_Date_Key"] = pd.to_numeric(
    fact_sales["Ship_Date_Key"], errors="coerce").astype("Int64")

# 8) Optional returns fact at returned-order grain
fact_returns = pd.DataFrame(columns=["Order_ID", "Order_Date_Key", "Returned"])
if RETURNS_PATH.exists():
    ret = normalise_columns(pd.read_excel(RETURNS_PATH, sheet_name=0))
    require_columns(ret, ["Order ID"], "Returns")
    ret["Order ID"] = ret["Order ID"].astype("string").str.strip()
    if "Returned" in ret.columns:
        ret["Returned"] = ret["Returned"].astype("string").str.strip()
    else:
```

```
        ret["Returned"] = "Yes"

    ret = ret.dropna(subset=["Order ID"]).drop_duplicates(subset=["Order ID"])
    order_dates = orders[["Order ID", "Order Date"]].drop_duplicates("Order ID")
    ret = ret.merge(order_dates, on="Order ID", how="left", validate="one_to_one")
    ret["Order_Date_Key"] = pd.to_datetime(
        ret["Order Date"], errors="coerce").dt.strftime("%Y%m%d")
    ret["Order_Date_Key"] = pd.to_numeric(
        ret["Order_Date_Key"], errors="coerce").astype("Int64")
    fact_returns = ret.rename(columns={"Order ID": "Order_ID"})[
        ["Order_ID", "Order_Date_Key", "Returned"]
    ]

# 9) Export
tables = {
    "dim_product": dim_product,
    "dim_customer": dim_customer,
    "dim_order": dim_order,
    "dim_date": dim_date,
    "fact_sales": fact_sales,
    "fact_returns": fact_returns
}
for name, frame in tables.items():
    frame.to_csv(OUT / f"{name}.csv", index=False)

quality.extend([
    f"Clean sales lines: {len(fact_sales)}",
    f"Products: {len(dim_product)}",
    f"Customers: {len(dim_customer)}",
    f"Orders: {len(dim_order)}",
    f"Date rows: {len(dim_date)}",
    f"Returned orders: {len(fact_returns)}"
])
REPORT.write_text("\\n\\n".join(quality), encoding="utf-8")
print("Prepared tables written to:", OUT)
print("\\n".join(quality[-6:]))
```

Note on date parsing: if the source uses ambiguous date strings (for example, 04/05/2025), confirm whether it means 4 May or April 5 before parsing. Do not use `dayfirst=True` unless the source format requires it.

5.3 Save a trained machine-learning model? (Optional extension)

This is a descriptive analytics project, not an ML project. You do not need to train a model to complete it. If you later add a forecasting or classification model, save it with joblib and document the feature schema and model version.

```
import joblib
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
joblib.dump(model, "output/model.joblib")

# Load later:
loaded_model = joblib.load("output/model.joblib")
predictions = loaded_model.predict(X_test)
```

Only include this section in your project if you actually build, evaluate and validate a model. Do not claim that a model was trained merely because the save/load example appears here.

6. Data model and table definitions

Recommended dimensional design

This design is a small fact constellation: Fact_Sales and Fact_Returns share common dimensions. Each dimension has a unique key, and facts contain foreign keys to those dimensions.

Table	Primary key	Important columns
Dim_Product	Product_ID	Product_Name, Category, Sub_Category
Dim_Customer	Customer_ID	Customer_Name, Segment, Country, City, State, Region
Dim_Order	Order_ID	Order identifier
Dim_Date	Date_Key	Date, Year, Quarter, Month_Number, Month_Name, Year_Month

Table	Primary key	Important columns
Fact_Sales	Sales_ID	Order_ID, Order_Date_Key, Ship_Date_Key, Customer_ID, Product_ID, Sales, Quantity, Discount, Pr
Fact_Returns	Order_ID	Order_Date_Key, Returned

Relationship diagram

Dim_Product (1) ■■■ (*) Fact_Sales (*) ■■■ (1) Dim_Customer
Dim_Date (1) ■■■ (*) Fact_Sales and Dim_Date (1) ■■■ (*) Fact_Returns
Dim_Order (1) ■■■ (*) Fact_Sales and Dim_Order (1) ■■■ (*) Fact_Returns

Avoid a direct Fact_Sales-to-Fact_Returns relationship. Let shared dimensions filter both fact tables. Use single-direction filtering from dimensions to facts in Power BI.

Date relationships

• Dim_Date[Date_Key] to Fact_Sales[Order_Date_Key]: active relationship.

• Dim_Date[Date_Key] to Fact_Sales[Ship_Date_Key]: inactive relationship; use USERELATIONSHIP for ship-date measures.

• Dim_Date[Date_Key] to Fact_Returns[Order_Date_Key]: active relationship when order date is available.

• Dim_Order[Order_ID] to both fact tables: one-to-many.

If Customer or Product attributes change historically, a simple one-row-per-ID dimension represents current attributes only. For historical reporting, research Slowly Changing Dimensions (SCD Type 2) rather than arbitrarily choosing one record.

7. SQL: create, load and analyse

7.1 Create the SQLite database

```
# scripts/03_load_sqlite.py
from pathlib import Path
import sqlite3
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
PROCESSED = ROOT / "data" / "processed"
DB = ROOT / "database" / "superstore.db"
DB.parent.mkdir(parents=True, exist_ok=True)

conn = sqlite3.connect(DB)
cur = conn.cursor()

cur.executescript("""
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS Fact_Returns;
DROP TABLE IF EXISTS Fact_Sales;
DROP TABLE IF EXISTS Dim_Order;
DROP TABLE IF EXISTS Dim_Date;
DROP TABLE IF EXISTS Dim_Customer;
DROP TABLE IF EXISTS Dim_Product;

CREATE TABLE Dim_Product (
    Product_ID TEXT PRIMARY KEY,
    Product_Name TEXT,
    Category TEXT,
    Sub_Category TEXT
);

CREATE TABLE Dim_Customer (
    Customer_ID TEXT PRIMARY KEY,
    Customer_Name TEXT,
    Segment TEXT,
    Country TEXT,
    City TEXT,
    State TEXT,
```

```
    Region TEXT
);

CREATE TABLE Dim_Order (
    Order_ID TEXT PRIMARY KEY
);

CREATE TABLE Dim_Date (
    Date_Key INTEGER PRIMARY KEY,
    Date TEXT NOT NULL,
    Year INTEGER,
    Quarter TEXT,
    Month_Number INTEGER,
    Month_Name TEXT,
    Year_Month TEXT,
    Day INTEGER,
    Day_Name TEXT
);

CREATE TABLE Fact_Sales (
    Sales_ID INTEGER PRIMARY KEY,
    Order_ID TEXT NOT NULL,
    Order_Date TEXT,
    Ship_Date TEXT,
    Order_Date_Key INTEGER,
    Ship_Date_Key INTEGER,
    Ship_Mode TEXT,
    Customer_ID TEXT NOT NULL,
    Product_ID TEXT NOT NULL,
    Sales REAL,
    Quantity INTEGER,
    Discount REAL,
    Profit REAL,
    FOREIGN KEY (Order_ID) REFERENCES Dim_Order(Order_ID),
    FOREIGN KEY (Order_Date_Key) REFERENCES Dim_Date(Date_Key),
    FOREIGN KEY (Customer_ID) REFERENCES Dim_Customer(Customer_ID),
    FOREIGN KEY (Product_ID) REFERENCES Dim_Product(Product_ID)
);

CREATE TABLE Fact_Returns (
    Order_ID TEXT PRIMARY KEY,
    Order_Date_Key INTEGER,
    Returned TEXT,
    FOREIGN KEY (Order_ID) REFERENCES Dim_Order(Order_ID),
    FOREIGN KEY (Order_Date_Key) REFERENCES Dim_Date(Date_Key)
);
""")

# Load dimensions first so foreign keys can be checked.
load_order = [
    ("dim_product", "Dim_Product"),
    ("dim_customer", "Dim_Customer"),
    ("dim_order", "Dim_Order"),
    ("dim_date", "Dim_Date"),
    ("fact_sales", "Fact_Sales"),
    ("fact_returns", "Fact_Returns")
]
for csv_name, table_name in load_order:
    df = pd.read_csv(PROCESSED / f"{csv_name}.csv")
    # Empty optional fact table is valid; skip insert if it has no rows.
    if not df.empty:
        df.to_sql(table_name, conn, if_exists="append", index=False)

conn.commit()
print("Database created:", DB)
print("Foreign key check:", cur.execute("PRAGMA foreign_key_check").fetchall())
print("Sales rows:", cur.execute("SELECT COUNT(*) FROM Fact_Sales").fetchone()[0])
conn.close()
```

The script drops and recreates tables each run, so it is suitable for a reproducible learning project, not a production database. Back up any database you need before rerunning it.

7.2 Example analytical SQL queries

```
-- Q1. Sales, profit and margin by category
SELECT p.Category,
       ROUND(SUM(f.Sales), 2) AS Total_Sales,
       ROUND(SUM(f.Profit), 2) AS Total_Profit,
       ROUND(100.0 * SUM(f.Profit) / NULLIF(SUM(f.Sales), 0), 2) AS Margin_Pct
FROM Fact_Sales f
JOIN Dim_Product p ON p.Product_ID = f.Product_ID
GROUP BY p.Category
ORDER BY Total_Sales DESC;
```

```sql
-- Q2. Monthly sales and profit
SELECT d.Year_Month,
       ROUND(SUM(f.Sales), 2) AS Sales,
       ROUND(SUM(f.Profit), 2) AS Profit
FROM Fact_Sales f
JOIN Dim_Date d ON d.Date_Key = f.Order_Date_Key
GROUP BY d.Year_Month
ORDER BY d.Year_Month;

-- Q3. Top 10 customers by sales
SELECT c.Customer_ID, c.Customer_Name,
       ROUND(SUM(f.Sales), 2) AS Sales,
       ROUND(SUM(f.Profit), 2) AS Profit
FROM Fact_Sales f
JOIN Dim_Customer c ON c.Customer_ID = f.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Sales DESC
LIMIT 10;

-- Q4. Top and bottom sub-categories by profit
SELECT p.Category, p.Sub_Category,
       ROUND(SUM(f.Profit), 2) AS Profit
FROM Fact_Sales f
JOIN Dim_Product p ON p.Product_ID = f.Product_ID
GROUP BY p.Category, p.Sub_Category
ORDER BY Profit;

-- Q5. Return rate (order-level)
SELECT
    (SELECT COUNT(DISTINCT Order_ID) FROM Fact_Returns) AS Returned_Orders,
    (SELECT COUNT(DISTINCT Order_ID) FROM Fact_Sales) AS Total_Orders,
    ROUND(
      100.0 * (SELECT COUNT(DISTINCT Order_ID) FROM Fact_Returns)
      / NULLIF((SELECT COUNT(DISTINCT Order_ID) FROM Fact_Sales), 0), 2
    ) AS Return_Rate_Pct;

-- Q6. Loss-making product lines
SELECT p.Product_Name, p.Category, p.Sub_Category,
       ROUND(SUM(f.Sales), 2) AS Sales,
       ROUND(SUM(f.Profit), 2) AS Profit
FROM Fact_Sales f
JOIN Dim_Product p ON p.Product_ID = f.Product_ID
GROUP BY p.Product_ID, p.Product_Name, p.Category, p.Sub_Category
HAVING SUM(f.Profit) < 0
ORDER BY Profit ASC;
```

SQL concepts demonstrated

• DDL: CREATE, DROP, ALTER (table structure).

• DML: INSERT, UPDATE, DELETE (data changes).

• SELECT, WHERE, GROUP BY, HAVING, ORDER BY and LIMIT.

• INNER JOIN / LEFT JOIN using primary-key and foreign-key relationships.

• Aggregate functions: SUM, AVG, COUNT, COUNT DISTINCT pattern, MIN and MAX.

• NULL handling with NULLIF and COALESCE; constraints and referential integrity.

8. Power BI: import and model

8.1 Connect to the data

• Open Power BI Desktop → Get Data → Text/CSV (or Excel) and load the six processed CSV files.

• Alternatively connect to the SQLite database using the available SQLite connector/ODBC driver in your environment.

• Prefer loading the prepared dimension and fact tables rather than importing the raw source and rebuilding all transformations independently.

• Use Transform Data to verify data types: IDs as Text (except numeric surrogate/date keys), dates as Date, measures as Decimal/Whole Number.

• Click Close & Apply.

8.2 Create relationships in Model view

From (one side)	To (many side)	Cardinality / status
Dim_Product[Product_ID]	Fact_Sales[Product_ID]	1:* active
Dim_Customer[Customer_ID]	Fact_Sales[Customer_ID]	1:* active
Dim_Order[Order_ID]	Fact_Sales[Order_ID]	1:* active
Dim_Date[Date_Key]	Fact_Sales[Order_Date_Key]	1:* active
Dim_Date[Date_Key]	Fact_Sales[Ship_Date_Key]	1:* inactive
Dim_Order[Order_ID]	Fact_Returns[Order_ID]	1:* active
Dim_Date[Date_Key]	Fact_Returns[Order_Date_Key]	1:* active

Use single-direction cross-filtering from dimension to fact. Do not connect Fact_Sales directly to Fact_Returns. Check that dimension keys are unique and that there are no unmatched fact foreign keys.

8.3 Date table

Select Dim_Date in Power BI → Table tools → Mark as date table → choose Dim_Date[Date]. Use the actual Date column (not the integer key). Sort Month_Name by Month_Number and use Year_Month for chronological monthly visuals.

For a standard time-intelligence setup, use a continuous calendar covering all dates in the fact data, with one row per date and no gaps. The Python script creates this calendar.

9. DAX measures library

Create a dedicated Measures table if desired (Home → Enter data, create a one-column placeholder, then hide the placeholder column). Add the following measures using Modeling → New measure. Names must match the table names in your model.

```
-- Core KPIs
Total Sales = SUM(Fact_Sales[Sales])

Total Profit = SUM(Fact_Sales[Profit])

Total Quantity = SUM(Fact_Sales[Quantity])

Total Orders = DISTINCTCOUNT(Fact_Sales[Order_ID])

Total Customers = DISTINCTCOUNT(Fact_Sales[Customer_ID])

Total Products = DISTINCTCOUNT(Dim_Product[Product_ID])

Profit Margin % = DIVIDE([Total Profit], [Total Sales], 0)

Average Order Value = DIVIDE([Total Sales], [Total Orders], 0)

Average Discount = AVERAGE(Fact_Sales[Discount])

Profit per Order = DIVIDE([Total Profit], [Total Orders], 0)

-- Returns: order-level only
Returned Orders = DISTINCTCOUNT(Fact_Returns[Order_ID])

Return Rate % = DIVIDE([Returned Orders], [Total Orders], 0)

-- Time intelligence
Previous Year Sales =
CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Dim_Date[Date]))

Sales Growth % =
DIVIDE([Total Sales] - [Previous Year Sales], [Previous Year Sales], 0)

YTD Sales =
TOTALYTD([Total Sales], Dim_Date[Date])

Previous Month Sales =
CALCULATE([Total Sales], DATEADD(Dim_Date[Date], -1, MONTH))

MoM Growth % =
```

```
DIVIDE([Total Sales] - [Previous Month Sales], [Previous Month Sales], 0)

-- Ship-date analysis (inactive relationship)
Sales by Ship Date =
CALCULATE(
    [Total Sales],
    USERELATIONSHIP(Dim_Date[Date_Key], Fact_Sales[Ship_Date_Key])
)

-- Profitability diagnostics
Loss-Making Sales Lines =
CALCULATE(
    COUNTROWS(Fact_Sales),
    Fact_Sales[Profit] < 0
)

Loss-Making Sales Amount =
CALCULATE(
    [Total Sales],
    Fact_Sales[Profit] < 0
)

-- Ranking: rank products by sales
Product Sales Rank =
RANKX(
    ALL(Dim_Product[Product_Name]),
    [Total Sales],
    ,
    DESC,
    DENSE
)
```

Customer measures

```
Repeat Customers =
COUNTROWS(
    FILTER(
        VALUES(Dim_Customer[Customer_ID]),
        CALCULATE(DISTINCTCOUNT(Fact_Sales[Order_ID])) > 1
    )
)

Repeat Customer Rate % =
DIVIDE([Repeat Customers], [Total Customers], 0)

Sales per Customer =
DIVIDE([Total Sales], [Total Customers], 0)
```

Format Sales, Profit, AOV and Sales per Customer as currency; margin, return rate, growth and discount as percentages; orders, customers, products and quantity as whole numbers. Use DIVIDE rather than `/` to handle zero denominators safely.

Time intelligence caveat: SAMEPERIODLASTYEAR, DATEADD and TOTALYTD rely on a correctly configured date table and a meaningful date filter context. Test them against a known period and ensure the date table covers the complete analysis period.

10. Dashboard pages and visual specifications

Build five report pages. Keep the same slicer conventions and colour meanings across pages. The visuals below are suggested designs; choose visuals based on the question rather than adding charts just to fill space.

Page 1 — Executive Summary

Area	Visual	Fields / measure
Top KPI row	Card visuals	Total Sales; Total Profit; Total Orders; Total Customers; Profit Margin %; Return Rate %
Main trend	Line chart	X: Dim_Date[Year_Month]; Y: Total Sales; optional second line: Total Profit
Geography	Bar chart or filled map (if geography is available)	Axis: Region; Values: Total Sales; tooltip: Total Profit and margin
Category performance	Clustered bar chart	Axis: Category; Values: Total Sales and Total Profit
Filters	Slicers	Year, Region, Segment, Category

Purpose: answer “How is the business performing?” quickly. Avoid putting too many small visuals on the first page.

Page 2 — Sales Performance

Visual	Fields / measures	Question answered
Line chart	Year_Month; Total Sales	How does sales trend over time?
Clustered column	Region; Total Sales and Total Profit	Which regions perform best?
Bar chart	Ship_Mode; Total Sales	How does shipping mode mix look?
Cards	AOV; Total Quantity; Sales Growth %; MoM Growth %	What are order value and growth?
Matrix	Year > Quarter > Month; Sales, Profit, Orders	How does performance break down over time?

Page 3 — Product and Category

Visual	Fields / measures	Use
Treemap	Category, Sub_Category; Total Sales	Show sales composition
Bar chart	Product_Name; Total Sales; Top N 10 filter	Top products by sales
Bar chart	Sub_Category; Total Profit; sort ascending	Find weakest sub-categories
Scatter chart	X: Average Discount; Y: Total Profit; Details: Product_Name; Size: Total Sales	Explore discount/profit relationship
Matrix	Category > Sub_Category; Sales, Profit, Margin	Compare product hierarchy

For scatter charts, ensure each dot represents a meaningful entity (such as product or sub-category). Aggregated averages can hide variation; explain the aggregation in the report.

Page 4 — Customer Analysis

Visual	Fields / measures	Use
Bar chart	Customer_Name; Total Sales; Top N 10	Highest sales customers
Donut or bar	Segment; Total Sales	Sales by segment
Map or bar	State / Region; Total Customers and Sales	Customer geography
Cards	Total Customers; Repeat Customers; Repeat Customer Rate %; Sales per Customer	Customer KPIs
Detail table	Customer, Orders, Sales, Profit	Customer-level review

Page 5 — Returns and Profitability

Visual	Fields / measures	Use
Cards	Returned Orders; Return Rate %; Total Orders	Return overview
Bar chart	Region or Segment; Returned Orders	Where returned orders are concentrated
Table	Returned Order ID and order date	Operational review
Bar chart	Sub_Category; Total Profit; sort ascending	Loss-making areas
Scatter chart	Discount vs Profit; Details: Product or Sub-category	Investigate discount/profit patterns

A return flag attached to an order does not prove that every product line in that order was returned. Label the page and measures as order-level returns unless the source gives returned line items.

11. Interactions, usability and design

Recommended report layout

• Use a 16:9 page. Keep a consistent title, navigation and slicer area.

• Place the most important KPI cards across the top; charts below; detail tables at the bottom or on drill-through pages.

• Use a restrained palette: dark blue for primary information, green for positive profit, red for negative profit, neutral grey for context.

• Use readable labels, meaningful titles and consistent currency/percentage formatting.

• Sort month names by Month_Number; use Year_Month for continuous chronological order.

• Add report-page tooltips for extra detail without cluttering the page.

Useful Power BI interactions

• Slicers: Year, Region, Segment, Category and Ship Mode. Use Edit interactions to control which visuals respond.

• Drill-down: Year → Quarter → Month; Category → Sub-Category.

• Drill-through: create a Product Detail page or Customer Detail page, passing Product_ID or Customer_ID.

• Tooltips: show Sales, Profit, Margin, Orders and Discount when hovering over a category or product.

• Bookmarks: create a reset-filters bookmark if it improves navigation.

• Conditional formatting: red for negative profit, green for positive profit; avoid relying on colour alone.

Titles and labels

Use question-led titles such as “Which sub-categories generate losses?” instead of generic labels such as “Chart 4”. Add units: Sales (■ or source currency), Profit (■ or source currency), Discount (%). Confirm the dataset currency before labelling it.

12. Validation and testing

A professional report must reconcile to the source and behave correctly under filters. Use the following checks before publishing.

Check	How to validate	Expected result
Row count	Compare raw cleaned line count to Fact_Sales rows	Differences documented
Unique sales key	Count duplicates in Sales_ID	Zero duplicates
Dimension keys	Check duplicates in each dimension primary key	Zero duplicates
Foreign keys	Check fact keys against dimensions	No unexplained orphans
Sales total	Compare Pandas, SQL and Power BI totals	Same within rounding
Profit total	Compare Pandas, SQL and Power BI totals	Same within rounding
Order count	Distinct Order_ID in each layer	Same result
Date range	Min/max order and ship dates vs Dim_Date	All fact dates covered
Returns rate	Distinct returned orders / distinct sales orders	Same filter scope and grain
Slicers	Select one year/region/category	All related visuals filter correctly
Time intelligence	Compare selected year with previous year	Correct date context

Useful SQL reconciliation

```
SELECT
  COUNT(*) AS Sales_Lines,
  COUNT(DISTINCT Order_ID) AS Orders,
  ROUND(SUM(Sales), 2) AS Sales_Total,
  ROUND(SUM(Profit), 2) AS Profit_Total
FROM Fact_Sales;
```

Useful Pandas reconciliation

```
print("Lines:", len(fact_sales))
print("Orders:", fact_sales["Order_ID"].nunique())
print("Sales:", round(fact_sales["Sales"].sum(), 2))
print("Profit:", round(fact_sales["Profit"].sum(), 2))
```

13. Business insights and recommendations

Do not invent findings before running the analysis. Use the templates below and fill them with your actual results.

Insight area	Evidence to calculate	Possible business action (only if evidence suppor
Regional performance	Sales, profit and margin by Region/State	Review pricing, product mix or coverage in weak areas
Product profitability	Profit by Category/Sub-Category/Product	Investigate costs, pricing and discounts for loss-makers
Discounting	Profit by discount band or scatter analysis	Set discount guardrails if high discounts correlate with lower profit
Customer value	Sales/profit/orders by customer and segment	Design retention or cross-sell programmes for valuable segments
Returns	Returned order rate by region/segment/time	Investigate operational causes; obtain line-level return reason data
Seasonality	Monthly sales and profit trend	Align inventory and promotions to recurring peaks

Insight writing formula

Observation → Evidence → Business implication → Recommendation.

Example template: “The [segment/region/category] contributed [X%] of sales but only [Y%] of profit during [period]. This suggests [interpretation]. The business should investigate [specific factor] and test [action], then monitor [metric].”

Distinguish correlation from causation. A discount/profit scatter chart may show association, but it does not prove discounts caused losses. Consider product mix, cost, customer type and order context.

14. Documentation and deliverables

README.md template

```
# Superstore Retail Analytics

## Business problem
Analyse retail sales, profitability, customers, products, shipping and
order-level returns to support business decisions.

## Tools
Python, Pandas, NumPy, SQLite, SQL, Power BI Desktop.

## Repository structure
See the folder tree in this project guide.

## How to run
1. Place source workbook(s) in data/raw/.
2. Install dependencies from requirements.txt.
3. Run: python scripts/01_profile_data.py
4. Run: python scripts/02_prepare_data.py
5. Run: python scripts/03_load_sqlite.py
6. Open powerbi/Superstore_Analytics.pbix and refresh.

## Data model
Fact_Sales is one row per order line. Fact_Returns is one row per
returned order when the source provides only an order-level flag.
Dimensions: Date, Product, Customer and Order.
```

```
## Measures
Sales, Profit, Margin, Orders, Customers, AOV, Growth and order-level
Return Rate. See the DAX measure documentation.

## Limitations
Document missing values, source currency, date assumptions, return
data grain and any excluded records.

## Findings
Add validated business insights after completing the analysis.
```

requirements.txt

```
pandas>=2.0
numpy>=1.24
openpyxl>=3.1
sqlalchemy>=2.0
matplotlib>=3.7
seaborn>=0.12
```

Data dictionary template

Field	Definition	Type	Rule / notes
Sales_ID	Unique sales line identifier	Integer	Unique, not null
Order_ID	Retail order identifier	Text	Repeats across order lines
Order_Date_Key	Integer key for order date	Integer	YYYYMMDD
Customer_ID	Customer identifier	Text	FK to Dim_Customer
Product_ID	Product identifier	Text	FK to Dim_Product
Sales	Sales amount for line	Decimal	Source currency
Profit	Profit amount for line	Decimal	Can be negative
Discount	Discount fraction	Decimal	Verify source scale
Returned	Order-level return flag	Text/Boolean	Only when source provides it

Final deliverables

• Raw source files preserved in data/raw/ (do not publish sensitive data).

• Python scripts and requirements.txt.

• Six processed tables in CSV format.

• SQLite database and reusable SQL analysis queries.

• Power BI PBIX with model, measures, report pages and working interactions.

• README, data dictionary, data model notes, data-quality report and validated insights.

• Optional screenshots or a short presentation demonstrating the report.

15. Interview explanation and resume bullets

90-second project explanation

“I completed an end-to-end retail analytics project using the Sample Superstore dataset. I first profiled the raw Orders and optional Returns files using Python and Pandas, checked missing values, duplicates, data types and key consistency, and transformed the data into customer, product, order and date dimensions with sales and returns fact tables. I loaded the prepared tables into SQLite, created primary and foreign key relationships, and wrote SQL queries using joins and aggregations to analyse sales, profit, customer and product performance. In Power BI, I built a dimensional model, created DAX measures for sales, profit margin, order count, average order value, growth and order-level return rate, and developed interactive report pages for executive summary, sales, products, customers and

returns. I reconciled key metrics between Python, SQL and Power BI and documented data limitations and business recommendations."

Resume bullets — use only after completing the work

• Built an end-to-end retail analytics pipeline using Python (Pandas) to profile, clean, validate and transform transactional sales data into reusable dimension and fact tables.

• Designed a relational SQLite database with primary/foreign key constraints and wrote SQL queries using joins, aggregations and subqueries to analyse sales and profitability.

• Developed an interactive Power BI report with a dimensional data model, DAX KPIs, time-intelligence measures, drill-downs and slicers across sales, product, customer and returns analysis.

• Performed reconciliation and data-quality checks across Python, SQL and Power BI; documented assumptions, data grain and order-level returns limitations.

Likely interview questions

• Why did you create fact and dimension tables? — To separate measurable transactions from descriptive attributes and simplify analysis.

• Why is Order ID not the sales fact primary key? — One order can contain multiple product lines; Row ID identifies the line.

• Why use DISTINCTCOUNT for orders? — The same Order ID may appear on several sales lines.

• What is a star schema? — A fact table connected to denormalised descriptive dimensions; this project has two facts sharing dimensions, so it is a small fact constellation.

• Why not connect the two fact tables directly? — Their grains differ; shared dimensions provide consistent filtering without ambiguous fact-to-fact joins.

• How did you calculate return rate? — Distinct returned orders divided by distinct sales orders in the same filter context, based on order-level return data.

• How did you validate the report? — Reconciled row counts, sales, profit and order counts across Pandas, SQL and Power BI, and tested relationships and slicers.

• What would you improve with more data? — Add return reasons, returned line items, product costs, targets, inventory and customer acquisition data.

16. Troubleshooting and final checklist

Common issues

Problem	Likely cause / fix
File not found	Check the exact filename, extension and folder; run from project root or use Path(__file__).
Worksheet not found	Print pd.ExcelFile(path).sheet_names and choose the correct sheet.
Date becomes NaT	Inspect source format, locale and invalid values; set an explicit format if consistent.
Duplicate dimension key	Inspect conflicting attributes; do not drop duplicates without a business rule.
Foreign-key insert fails	Load dimensions before facts; check unmatched IDs and data types.
Power BI many-to-many relationship	Dimension key is not unique or wrong key selected; fix dimension grain/key.
Month sorting alphabetically	Sort Month_Name by Month_Number or use Year_Month.
Previous-year DAX blank/wrong	Check date table continuity, date-table marking and selected date range.
Return rate exceeds expectation	Check duplicate returned orders, denominator scope, date filters and whether data is order-level.

Problem	Likely cause / fix
Power BI totals differ from SQL	Check relationships, filters, duplicate rows, numeric types and measure definitions.

Project completion checklist

■ Source files preserved and dataset columns understood

■ Data grain documented for each fact table

■ Python profiling completed and quality report saved

■ Cleaning rules reviewed; no unexplained rows silently removed

■ Dimensions and facts exported to CSV

■ Date dimension is continuous and covers required dates

■ SQLite tables created with PK/FK constraints

■ SQL analytical queries run and results checked

■ Power BI relationships match the documented model

■ Dim_Date marked as date table and month sorting fixed

■ All DAX measures formatted and validated

■ Five report pages built with consistent layout

■ Slicers, drill-down, tooltips and drill-through tested

■ Sales, profit, order counts and returns reconciled

■ Insights written from actual results, not assumptions

■ README, data dictionary, limitations and PBIX saved

You are ready to present the project when you can explain the source grain, every transformation, the relationship design, the meaning of each KPI and at least three evidence-backed business insights.

Appendix A — Key definitions

Term	Plain-English meaning
Data cleaning	Finding and correcting or handling quality problems in source data.
Data transformation	Changing data structure or format to make it useful for analysis.
Fact table	Table containing measurable business events at a defined grain.
Dimension table	Table containing descriptive attributes used to slice facts.
Primary key	Column(s) that uniquely identify a row in a table.
Foreign key	Column(s) that reference a key in another table.
Star schema	A central fact table linked to dimension tables.
Fact constellation	Multiple fact tables sharing common dimensions.
DAX	Data Analysis Expressions, the formula language used in Power BI.
Measure	A calculation evaluated in the current filter context.
Filter context	The filters active when a DAX measure is evaluated.
Drill-through	Navigate to a detail page filtered to a selected entity.

Term	Plain-English meaning
AOV	Average Order Value = Sales / distinct Orders.